5. 1 Implementation of Data Structure using Java Class:

5.1.1 Concepts of Singly and Singly Circular linked list.
5.1.2 Singly Linked List: Create, Traverse, Insert and Delete node.
5.1.3 Singly Circular Linked List: Create, Traverse, Insert and Delete node.

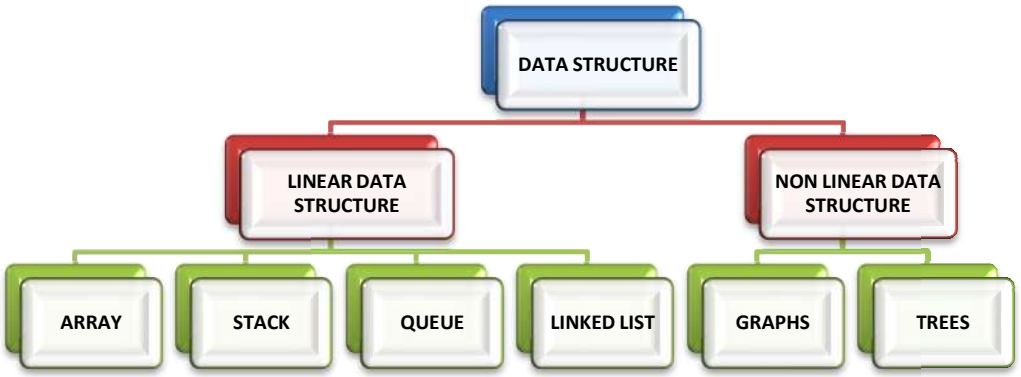
5. 1 Implementation of Data Structure using Java Class

Concept to Data Structure

A data structure is a way to organize, store, and manage data so it can be used efficiently. It is like a container for data.

Types of Data Structures:

- **Linear**: Elements are arranged sequentially.
    - Examples: **Array**, **Linked List**, **Stack**, **Queue**
- **Non-Linear**: Elements are arranged in a hierarchical or interconnected structure.
    - Examples: **Tree**, **Graph**



Concept of Singly Circular linked list

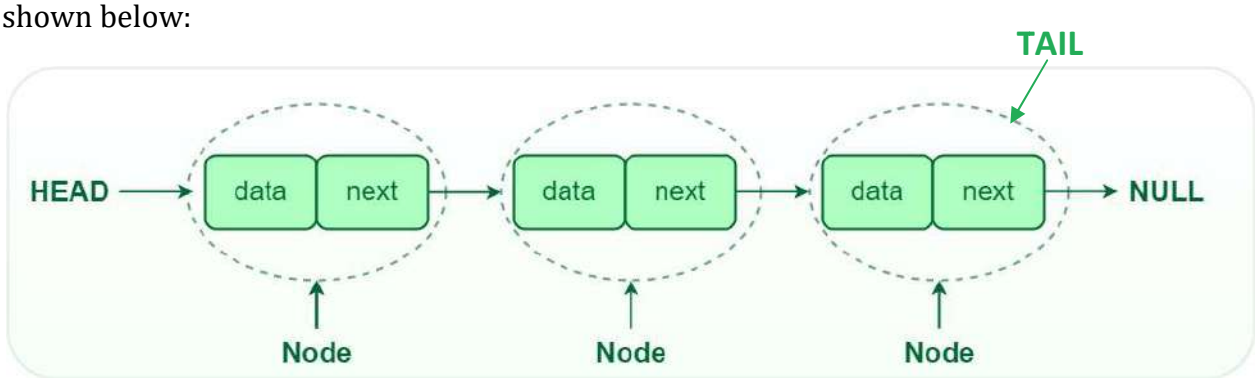
A **linked list** is a type of data structure where data is stored in a sequence of nodes, and each node has two parts:

Data: The value or information.

Pointer: A link to the next node in the list.

Unlike arrays, linked lists don't have a fixed size, and can be easily add or remove elements without shifting other elements around. However, It is needed to follow the links (pointers) from one node to the next to access the data, which can take more time than accessing an array element directly.

In short, a linked list is like a chain where each link (node) knows where the next one is as shown below:



Head and Tail: The linked list is accessed through the head node, which points to the first node in the list. The last node in the list points to NULL or null pointer, indicating the end of the list. This node is known as the tail node.

The real life example of Linked List is that of Railway Carriage. It starts from engine and then the coaches follow. Coaches can traverse from one coach to other, if they connected to each other.

Operations on Singly Linked List

Creation of Node

To create a Node in a singly linked list in Java, define a class that contains two components:

1. **Data**: Stores the actual value of the node.
2. **Next**: A reference (pointer) to the next node in the list.

Here's a basic code to create a Node class in Java for a singly linked list:

```
class Node {
   int data;  // Data stored in the node
   Node next; // Pointer to the next node

   // Constructor to create a new node
   Node(int d) {
      data = d;
      next = null;  // When a new node is created, its next is null
   }
}
```

Traversing the Node

Traversing a singly linked list means going through each node from the head (first node) to the last node (where the next pointer is null). During traversal, you can access and work with the data stored in each node.

```
// Method to traverse the linked list and print its elements

public void traverse() {
    Node current = head;  // Start from the head

    while (current != null) {
       System.out.print(current.data + " ");   // Print the data
       current = current.next;          // Move to the next node
    }

    System.out.println();
 }
```

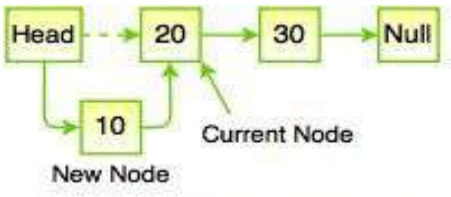
Insertion

Inserting a node into a singly linked list can be done in three different positions:

1. **At the beginning (head).**
2. **At the end (tail).**
3. **In the middle (after a specific node).**

1. Insert at the Beginning:

To insert a node at the beginning, we make the new node the head, and its next points to the current head.

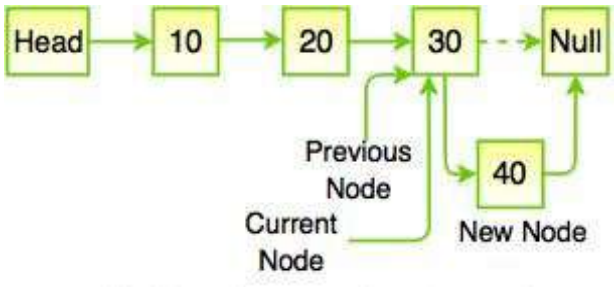



```
// Insert a new node at the beginning

  public void insertAtBeginning(int newData) {

    Node newNode = new Node(newData);
    newNode.next = head;  // New node points to current head
```

```
    head = newNode;      // Head now points to the new node

}
```

2. Insert at the End:

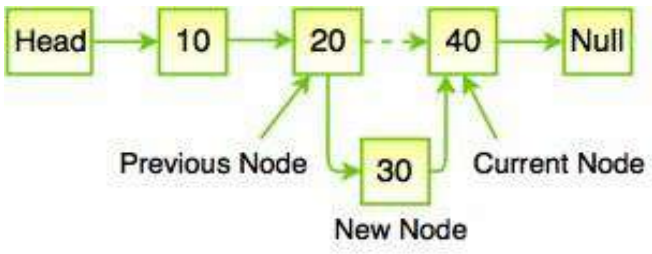
To insert at the end, we traverse the list until we reach the last node, then point the last node's next to the new node.




```
// Insert a new node at the end

  public void insertAtEnd(int newData) {

    Node newNode = new Node(newData);

    if (head == null) {   // If the list is empty
      head = newNode;
      return;

    }
```

3. Insert in the Middle (after a specific node):

To insert after a specific node, we find that node, and then insert the new node between it and the next one.



```
// Insert a new node after a given node

  public void insertAfterNode(Node prevNode, int newData)
  {
    if (prevNode == null) {  // Check if the given node is null

      System.out.println("The given previous node cannot be null.");
      return;
    }

    Node newNode = new Node(newData);
    newNode.next = prevNode.next;  // New node points to the next of the previous node
    prevNode.next = newNode;      // Previous node now points to the new node
  }
```

Deletion

Removing a node from a linked list requires adjusting the pointers of the neighboring nodes to bridge the gap left by the deleted node.

In a singly linked list, deleting a node involves different scenarios:

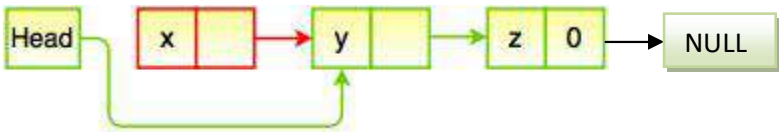
1. **Delete the head node (first node).**
2. **Delete a node with a specific value.**
3. **Delete the last node.**

1. Delete the head node

Sets head to head.next, removing the first node.

Fig. Deleting a node from the beginning

Solution is:




```
public void deleteHead()
 {
    if (head != null) {
```

```
        head = head.next;
    }
}
```

2. Delete a node with a specific value.

Traverses to find the node with the specified value, and then unlinks it from the list.

Fig. Deleting a node from the middle

```
public void deleteNode(int value)
{
    if (head == null) return;

    // If the head node holds the value
    if (head.data == value) {
        head = head.next;
        return;
    }

    Node temp = head;
    while (temp.next != null && temp.next.data != value) {
        temp = temp.next;
    }

    // If node with value found, unlink it
    if (temp.next != null) {
        temp.next = temp.next.next;
    }
}
```

3. Delete the last node.

Traverses to the second-to-last node and sets its next to null, removing the last node.

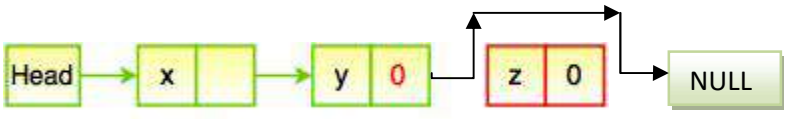


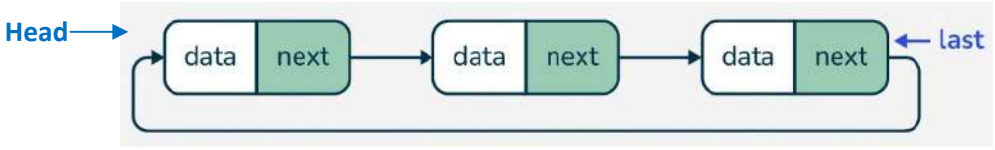
Fig. Deleting a node from the end

```
public void deleteLast() {
    if (head == null) return;

    // If only one node is present
    if (head.next == null) {
       head = null;
       return;
    }

    Node temp = head;
    while (temp.next.next != null) {
       temp = temp.next;
    }
    temp.next = null; // Unlink the last node
  }
```

5. 1.3 Singly Circular Linked List: Create, Traverse, Insert and Delete node.

Concept of Singly Circular Linked list

- A circular singly linked list is similar to a regular singly linked list, but with one key difference: the last node in a circular singly linked list points back to the head node, forming a circular structure.
- In a circular singly linked list, the next pointer of the last node doesn't point to null as in a normal singly linked list. Instead, it points back to the head of the list.
- Like a regular linked list, a circular singly linked list has a head node which serves as the entry point for traversal.
- Since the list is circular, traversing the list starting from the head will eventually loop back to the head.



Advantages:

- Circular linked lists are useful in scenarios where you want a data structure that can easily "wrap around" without reaching an endpoint, such as in round-robin scheduling or implementing a circular buffer.

Disadvantages:

- They are a bit more complex to implement than singly linked lists since you need to handle the circular reference properly.
- Insertions and deletions require more attention to maintain the circular connection.

Creation of Node in Circular Singly Linked List

To insert a node in empty circular linked list, creates a new node with the given data, sets its next pointer to point to itself, and updates the last pointer to reference this new node.

1. **Define the Node Class**
   - The Node class stores the data and a reference (next) to the next node.
   - Each node in the list has a data field and a next field that points to the next node.

```
class Node
 {
     int data;
     Node next;

     Node(int data)
     {
       this.data = data;
     }

 }
```

2. **Traversing through the node of circular linked list**

   Traversing and displaying nodes in a circular linked list can be achieved by starting from the head node and continuing to move to the next node until we reach back to the head

```
// Method to display the list

public void display()
 {
   Node current = head;

   if(head == null)
```

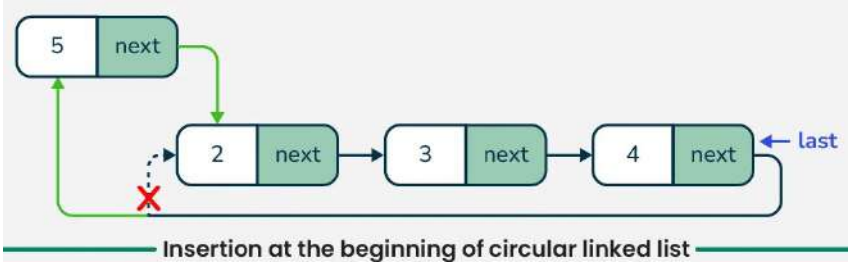
```
  {
   System.out.println("List is Empty");
  }
  else
  {
   do
   {
    System.out.println(" "+current.data);
    current=current.next;

   }while(current != head);
  }

 }
```

3. Insertion in a Circular Singly Linked List

Insertion in a Circular Singly Linked List can be done at various points: at the beginning, at the end, or at a specific position.

3.1 Inserting at the Beginning:

Update the head and adjust the last node's next pointer.

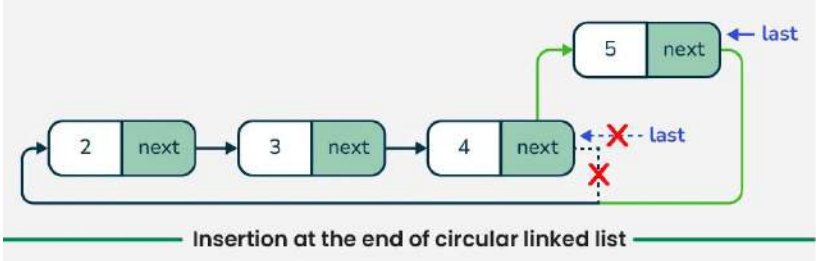



```
// Insert a node at the beginning

  public void insertBeginning(int data) {
     Node newNode = new Node(data);

     // If list is empty
     if (head == null) {
        head = newNode;
        tail = newNode;
        newNode.next = head;
     } else {
        newNode.next = head; // Link new node to current head
        head = newNode;      // Move head to new node
```

```
        tail.next = head;    // Circular link from tail to new head
    }
}
```

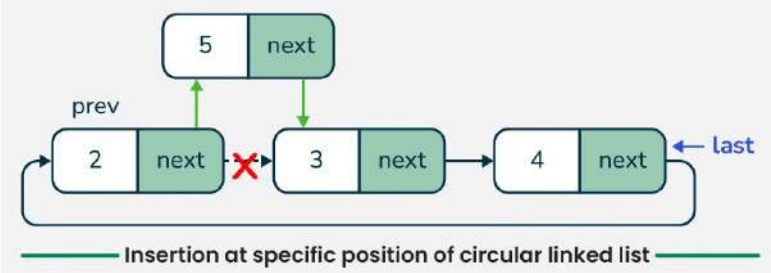
3.2 Inserting at the End:

Update the tail to point to the new node and make sure it connects back to head.




```
// Insert node at the end

public void insertAtEnd(int data) {
    Node newNode = new Node(data);

    if (head == null) {
        head = newNode;
        tail = newNode;
        newNode.next = head;
    } else {
        tail.next = newNode;  // Link current tail to new node
        tail = newNode;       // Move tail to the new node
        tail.next = head;     // Make it circular by pointing new tail to head
    }
}
```

3.3 Insertion at the specific position of circular linked list



```
// Method to add a node at a specific position

 public void insertAtPosition(int data, int position) {
     Node newNode = new Node(data);

     // Case 1: Insert at the head (beginning) of the list
     if (position == 1) {
        if (head == null) {
           head = newNode;
           tail = newNode;
           tail.next = head;  // Complete the circular link
        } else {
           newNode.next = head;
           head = newNode;
           tail.next = head;  // Update the tail's next pointer
        }
     } else {
        Node current = head;
        int count = 1;

        // Traverse to the node just before the desired position
        while (count < position - 1 && current.next != head) {
           current = current.next;
           count++;
        }

        // Insert the new node in the list
        newNode.next = current.next;
        current.next = newNode;

        // If inserting at the end, update the tail
        if (current == tail) {
           tail = newNode;
           tail.next = head;  // Maintain the circular link
        }
     }
  }
```

Explanation of the Code:

1. **Insert at the Head (Position = 1)**:
    - If the list is empty, initialize both head and tail with the new node and link tail.next to head.

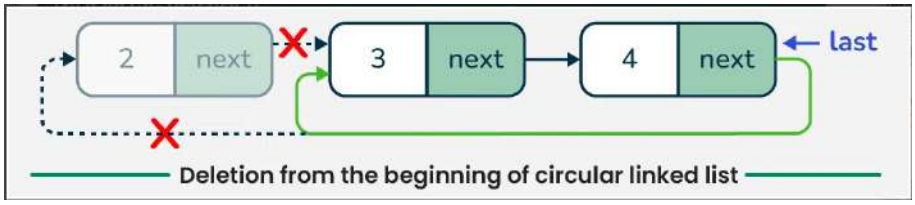
- If the list is not empty, update newNode.next to point to head, then set head to newNode. Finally, update tail.next to maintain the circular structure.

2. **Insert at a Specific Position (2, 3, ...)**:
   - Traverse to the node just before the desired position by iterating and keeping a count.
   - Insert the new node between current and current.next.
3. **Insert at the Tail**:
   - If position corresponds to the end, tail is updated to the new node, and the circular link is maintained by setting tail.next to head.

4. Delete node in Singly Circular linked list

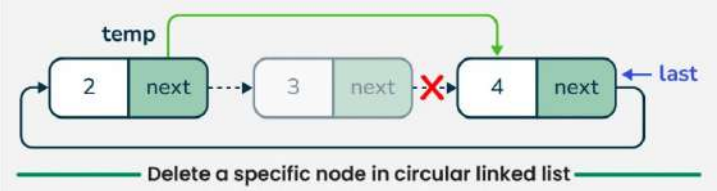
To delete a node in a singly circular linked list, following three primary cases:

1. **Deleting the head node.**



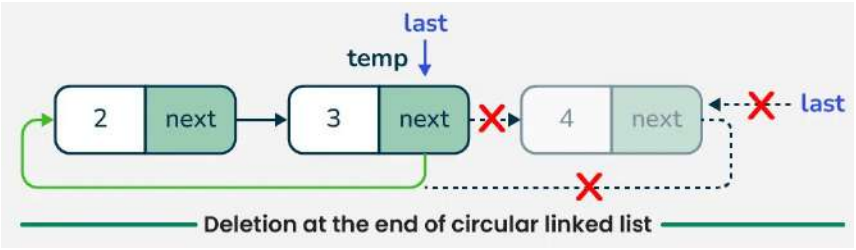

```
// Method to delete the head node
  public void deleteHead() {
     if (head == null) {
        System.out.println("List is empty, nothing to delete.");
        return;
     }
     if (head == tail) { // If there is only one node
        head = null;
        tail = null;
     } else {
        head = head.next; // Move head to the next node
        tail.next = head; // Update tail's next pointer to maintain the circular structure
     }
     System.out.println("Head node deleted.");
  }
```

2. Deleting the tail node.




```
// Method to delete the tail node
  public void deleteTail() {
    if (head == null) { // If the list is empty
      System.out.println("List is empty, nothing to delete.");
      return;
    }

    if (head == tail) { // If there's only one node
      head = null;
      tail = null;
    } else {
      Node current = head;

      // Traverse to the node just before the tail
      while (current.next != tail) {
        current = current.next;
      }

      current.next = head; // Update the next pointer to maintain the circular link
      tail = current;      // Update the tail to the new last node
    }
  }
```

3. Deleting a node at a specific position.



Here, there are the Node and CircularSinglyLinkedList classes and Implement a method deleteAtPosition(int position) to delete a node at a specific position.

```
// Delete a node at a specific position

  void deleteNodeAtPosition(int pos)
   {
     Node temp = head;
     Node prevNode = null;

     if(pos < 1 || pos > size){
        System.out.println("Invalid\n");
        return;
     }

     // delete the 1st node
     if(pos == 1){
        head = head.next;
        tail.next = head;
        System.out.println("Deleted: " + temp.data);
        size--;
        return;
     }

     // traverse to the pos'th node
    for(int i=1;i<pos;i++)
    {
        prevNode = temp;
        temp = temp.next;
    }

     // change prevNode node's next node to nth node's next node
     prevNode.next = temp.next;
     // delete this nth node
     System.out.println("Deleted: " + temp.data);
     size--;

   }
```